HW2

1. $X \in R^{n \times d}$ is mean centered (each feature has zero mean)

a. Projecting each point $x_i \in R^d$ onto $w \in R^d$ with $\|w\|_2 = 1$ producing $z_i = w^T x_i$. Show that variance of the projected data can be written as $\text{var}(z) = w^T \Sigma w$, where $\Sigma = \frac{1}{n} X^T X$ is sample covariance matrix

mean formula: $\bar{z} = \frac{1}{n}(\sum_i z_i)$

variance formula: $\text{Var} = \frac{1}{n}(\sum_i (z_i - \bar{z})^2)$

$\bar{z} = \frac{1}{n}(\sum w^T x_i)$

$\bar{z} = \sum w^T x_i / n$

$\bar{z} = w^T (\frac{\sum x_i}{n})$

$\bar{z} = 0$

$0 = w^T (\frac{\sum x_i}{n})$

$0 = (w^T (\frac{1}{n})) (\sum x_i)$

$0 = w^T \sum_{x_i} = \bar{z}$

$\frac{1}{n}(\sum_i (w^T x_i - 0)^2)$

$(\sum_i (w^T x_i)^2)/n$

$= (\sum_i (w^T (x_i x_i^T) w))/n$

$= w^T (\sum (x_i x_i^T))(\frac{1}{n}) w$

$= w^T \Sigma_i w$

$\boxed{\text{var}(z) = w^T \Sigma_i w}$

$w^T x_i \cdot w^T x_i =$

$w^T x_i \cdot (w^T x_i)^T$

$w^T x_i \cdot (x_i^T w)$

$w^T (x_i x_i^T) w$

b. PCA seeks the direction w that maximizes $w^T \Sigma_i w$ subject to $\|w\|_2 = 1$

; Use Lagrange Multiplier, show that the optimal solution satisfies $\Sigma_i w = \lambda w$

Lagrange multiplier: $L(x, \lambda) = f(x) - \lambda g(x)$

Σ_i = matrix = X

$f(x) = w^T \Sigma_i w$

$w^T \Sigma_i w - \lambda (w^T \Sigma w - 1)$

$\frac{d}{dx} = w^T \Sigma_i w - \lambda (w^T w - 1)$

$= w^T X w$ ← ignore

$\|w\|_2 = 1$ (norm of $w = 1$)

$\|w\|_2 = \sqrt{w_1^2 + w_2^2 + w_3^2 \ldots w_x^2} = 1$

$(\|w\|_2)^2 = (\sqrt{w^T w})^2 = 1$

$= w^T \cdot w = 1$

$w^T w = 1$

$0 = 2Xw - \lambda(2w)$

$2\Sigma w - \lambda 2w = 0$

$2(\Sigma w - \lambda w) = 0$

$\Sigma w - \lambda w = 0$

$\boxed{\Sigma w = \lambda w}$

b. ii. Explain why the eigenvector corresponding to the largest eigenvalue is selected as the first PC.

PCA components are chosen as they are the features that best explain the variance of the data. Since covariance measures the joint variability of 2 random variables, the eigenvalues & eigenvectors with the largest variance are identified as the first component

c. PCA can also be viewed as $\sum_{i=1}^{n} \|x_i - \tilde{x}_i\|_2^2$, where $\tilde{x}_i$ is the projection of x_i onto the lower-dimensional subspace. Explain why this makes PCA sensitive to global structure of the data. Discuss how distances between far points influence the solution.

PCA functions on the data's spread. When many points are far from each other, the variance increases. The reconstruction error is calculated by summing the distance between the original point and its projection. This means that points where the distance to its projection is larger, dominate the error. PCA tries to minimize that by keeping the larger structure of the data intact but the local accuracy diminishes. This is especially the issue with multidimensional non-linear data. By operating as linear structures, curves and folds in the data are deemed structureless and get collapsed.

d. t-SNE formula: $p(j|i) = \frac{\exp\left(-\|x_i - x_j\|^2/(2\sigma_i^2)\right)}{\sum_{k \neq i} \exp\left(-\|x_i - x_k\|^2/(2\sigma_i^2)\right)}$

Explain the intuition behind t-SNE.

Looking at the formula, we see similarities to the Gaussian function. This means that there is some similarities. What this means is that t-SNE uses a Gaussian PMF to figure out how similar 2 points are in the higher dimension, compares the point (x_i) to another point (x_k) and plots x_i in the lower dimension based on its probable relation to x_j and x_k.

1. e. In t-SNE, similarities in the low dimensional embedding is defined as:

$$q_{ij} = \frac{(1+\|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l}(1+\|y_k - y_l\|^2)^{-1}}$$

The low dimensional embedding is obtained by minimizing the Kullback-Leibler (KL) divergence:

$$\mathcal{L} = \sum_{i,j} p_{ij} \log \frac{p_{ij}}{q_{ij}} \leftarrow \text{loss function of t-SNE}$$

i. Explain how the weighting by p_{ij} affects which 2 points contribute most to the loss.

p_{ij} refers to the points in the higher-dimensional space and q_{ij} is the same pair in the lower dimensional spaces. When p_{ij} is larger than q_{ij}, it means the pair of points are closer in the higher dimension space but get pulled away in the lower space. What this means for the loss function, is that points that are very close to eachother in the higher will contribute more to the loss function.

ii. Explain why a small p_{ij} and a large q_{ij} would not incurr a big penalty.

We can rewrite the loss function as $\sum p_{ij} \log(p_{ij}) - \log(q_{ij})$. Since $\log(q_{ij})$ is large we would get a negative loss. But that is mitigated by the fact the small $\log(p_{ij})$ is being multiplied by p_{ij}, in a sense "balancing" itself out (like $4 \times \frac{1}{4} = 1$). Thus, the large $\log(q_{ij})$ value is "neutralized" by the "balanced" $p_{ij} \log(p_{ij})$ term, resulting in a smaller penalty.

iii. What would change if t-SNE minimized $KL(Q\|P)$ instead of $KL(P\|Q)$.

By focusing on $KL(Q\|P)$, what would happen is that rather than preserving local structures, t-SNE would preserve global structure. the loss function focuses on maintaing a high p_{ij}. In other words, keeping higher-dimension points close. When we flip the loss function, we focus on keeping the lower dimension points close, and also keeping far points in the higher dimension far away. This in turn doesn't preserve local structure, but global.

1. f. Why is PCA global and t-SNE local?

PCA aims to reduce dimensionality by maximizing the variance along the various axes, and minimizing the reconstruction error. As mentioned before, distance between points heavily influence PCA, so if 2 points are far, PCA aims to preserve that, thus PCA is global. T-SNE on the other hand is more local as it focuses on how close 2 points are in the higher dimension, and attempts to preserve that closeness in its loss function, thus making it more local.

Sources:

- medium.com/@Pavitkam-1k/dimensionality-reduction-t-sne-78658086460
- Van der Maaten, Laurens, and Geoffery Hinton. Visualizing Data using T-SNE
- AI was used for 2 for debugging purposes (Claude)

Question 2

```
import scimap
import anndata as ad
import matplotlib.pyplot as plt
from IPython.display import Image
import numpy as np
import random
import math
import pandas as pd
```

```
Running SCIMAP  2.3.5
```

Part A

i.

```
a_d = ad.read_h5ad("S2026HW2\sc_pca.h5ad")
a_d
```

```
AnnData object with n_obs × n_vars = 34162 × 0
    obs: 'merged_annotation'
    obsm: 'X_pca'
```

```
X_pca = a_d.obsm["X_pca"]
X_pca.shape[1]
```

```
30
```

```
a_d.obs.copy()
```

```
                                  merged_annotation
index
N51.LPA.TCATTTGGTCAGAAGC                     Plasma
N18.LPB.GGTAGTACACAGTC                       Plasma
N10.LPB.GTACGAACCTATTC                       Plasma
N11.LPB.TCCGAAGAAGATGA                       Plasma
N51.LPA.GTTACAGCACAGAGGT                     Plasma
...                                             ...
N46.LPB.TGCACCTCAGGATTGG           Immature Goblet
N46.LPB.TGGCGCAAGCTACCGC  Immature Enterocytes 1
N46.LPB.TGTGGTAGTCTGATTG           Immature Goblet
N46.LPB.TTGCGTCTCACCTCGT               Enterocytes
N46.LPB.TTTGGTTTCAATCACG           Immature Goblet

[34162 rows x 1 columns]
```

```
idx = []
for i in range(30):
```

```
    idx.append(f"PC{i+1}")
df = pd.DataFrame(index=idx)
df
```

```
Empty DataFrame
Columns: []
Index: [PC1, PC2, PC3, PC4, PC5, PC6, PC7, PC8, PC9, PC10, PC11, PC12,
PC13, PC14, PC15, PC16, PC17, PC18, PC19, PC20, PC21, PC22, PC23,
PC24, PC25, PC26, PC27, PC28, PC29, PC30]
```

```
adata = ad.AnnData(X=X_pca,obs=a_d.obs.copy(),var=df)
print(adata.obs, adata.var)
```

```
                                merged_annotation
index
N51.LPA.TCATTTGGTCAGAAGC                   Plasma
N18.LPB.GGTAGTACACAGTC                     Plasma
N10.LPB.GTACGAACCTATTC                     Plasma
N11.LPB.TCCGAAGAAGATGA                     Plasma
N51.LPA.GTTACAGCACAGAGGT                   Plasma
...                                           ...
N46.LPB.TGCACCTCAGGATTGG         Immature Goblet
N46.LPB.TGGCGCAAGCTACCGC  Immature Enterocytes 1
N46.LPB.TGTGGTAGTCTGATTG         Immature Goblet
N46.LPB.TTGCGTCTCACCTCGT             Enterocytes
N46.LPB.TTTGGTTTCAATCACG         Immature Goblet

[34162 rows x 1 columns] Empty DataFrame
Columns: []
Index: [PC1, PC2, PC3, PC4, PC5, PC6, PC7, PC8, PC9, PC10, PC11, PC12,
PC13, PC14, PC15, PC16, PC17, PC18, PC19, PC20, PC21, PC22, PC23,
PC24, PC25, PC26, PC27, PC28, PC29, PC30]
```

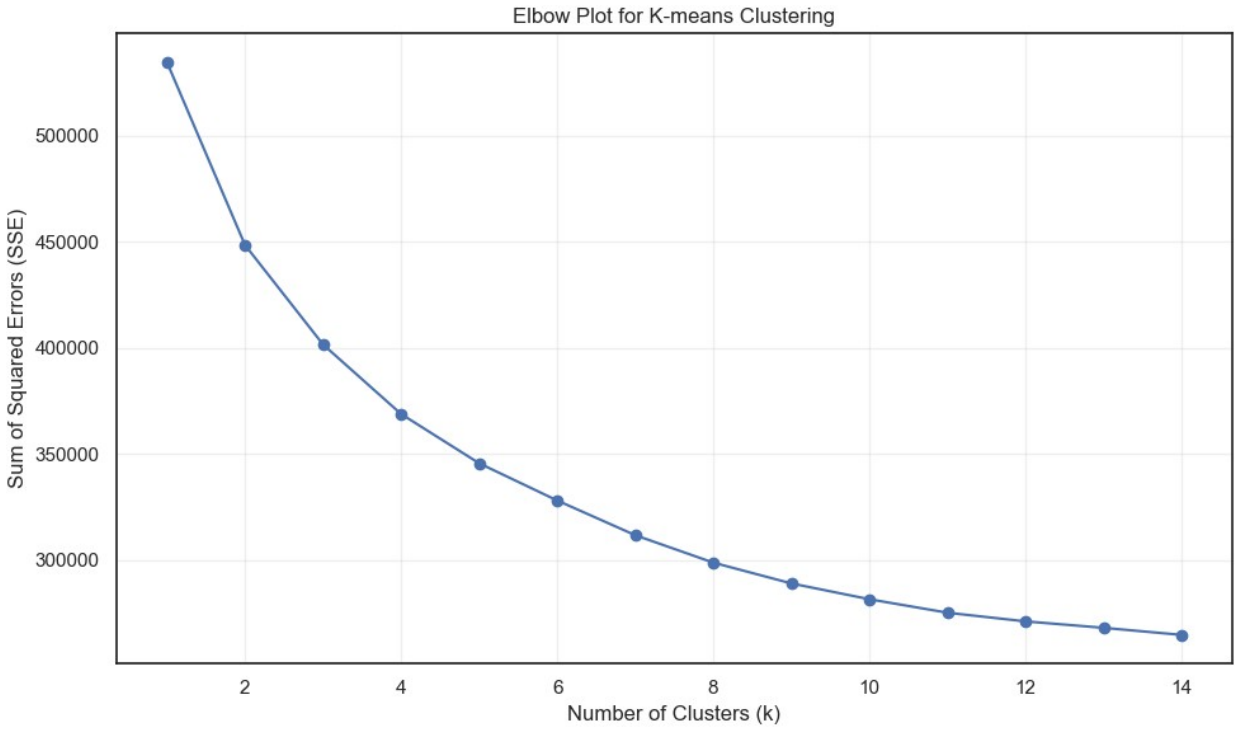
```
X = adata.X
sses = []

for k in range(1, 15):

scimap.tl.cluster(adata,method="kmeans",k=k,label=f"kmeans_{k}",use_ra
w = False,random_state=42)
    labels = adata.obs[f"kmeans_{k}"].astype(int).values
    centroids = np.array([X[labels == j].mean(axis=0)for j in
range(k)])
    sse_k = sum(np.linalg.norm(X[labels == j] - centroids[j],
axis=1).sum()for j in range(k))
    sses.append(sse_k)
sses
```

```
Kmeans clustering
Kmeans clustering
Kmeans clustering
```

```
Kmeans clustering
Kmeans clustering
Kmeans clustering
Kmeans clustering
Kmeans clustering
Kmeans clustering
Kmeans clustering
Kmeans clustering
Kmeans clustering
Kmeans clustering
Kmeans clustering
```

```
[534636.375,
 448495.453125,
 401646.6640625,
 368894.484375,
 345690.97265625,
 328151.68359375,
 311812.453125,
 298877.962890625,
 289080.962890625,
 281630.275390625,
 275282.1025390625,
 271200.8388671875,
 268167.6318359375,
 264830.0859375]
```

```
plt.figure(figsize=(10, 6))
plt.plot(range(1, 15), sses, 'bo-')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Sum of Squared Errors (SSE)')
plt.title('Elbow Plot for K-means Clustering')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Bases on the produced elbow plot, the elbow point appears to be between 10. This means that 5 is the most optimal number of clusters (the last point befroe the rate of decrease in SSE becomes approximately linear). It does not work too well on sc data however due to noise in the data espetially considering the diversity of DE genes. Additionally, like the plot above, it is difficult to discern where that elbow point actually as anoother scientist could argue the point is at 11 or 8.

ii.

```
# Set random seeds for reproducibility
random.seed(42)
np.random.seed(42)
scimap.tl.cluster(adata,method='kmeans',k=10,label='kmeans_10',use_raw
= False,random_state=42)

# Calculate SSE
X = adata.X
labels = adata.obs['kmeans_10'].astype(int).values
centroids = np.array([X[labels == j].mean(axis=0) for j in range(10)])
sse = sum(np.linalg.norm(X[labels == j] - centroids[j], axis=1).sum()
for j in range(10))

print(f"Final SSE with stable random_state=42: {sse:.2f}")
```

```
Kmeans clustering
Final SSE with stable random_state=42: 281630.28
```

iii.

```
def adjusted_rand_index(labels_true, labels_pred):
    contingency = np.zeros((len(np.unique(true_labels)),
len(np.unique(labels))), dtype=int)
    for i in range(len(true_labels)):
        contingency[true_labels[i], labels[i]] += 1
    a = np.sum([math.comb(n, 2) for n in np.sum(contingency, axis=1)])
    b = np.sum([math.comb(n, 2) for n in np.sum(contingency, axis=0)])
    ab = np.sum([math.comb(n, 2) for n in contingency.flatten() if n >
1])
    total_pairs = math.comb(len(true_labels), 2)
    ri = (ab + (total_pairs - a - b + ab)) / total_pairs
    expected_ri = (a * b) / total_pairs
    max_ri = (a + b) / 2
    ari = (ri - expected_ri) / (max_ri - expected_ri)
    return ari

true_labels =
adata.obs['merged_annotation'].astype('category').cat.codes.values

ari = adjusted_rand_index(true_labels, labels)
print(f"Adjusted Rand Index (ARI): {ari:.4f}")
```

```
Adjusted Rand Index (ARI): -0.0000

C:\Users\Trish\AppData\Local\Temp\ipykernel_27356\3703409219.py:10:
RuntimeWarning:

overflow encountered in scalar multiply
```

The ARI value of 0 can mean that the clusters are random and independent.

B

i.

```
scimap.tl.cluster(adata,method='leiden',nearest_neighbors=20,resolutio
n=0.1,label='leiden_0.1',random_state=42,use_raw=False,
phenograph_clustering_metric = 'Euclidean')
```

```
Leiden clustering

C:\Users\Trish\anaconda3\Lib\site-packages\scanpy\preprocessing\_pca\
__init__.py:384: ImplicitModificationWarning:

Setting element `.obsm['X_pca']` of view, initializing view as actual.

C:\Users\Trish\anaconda3\Lib\site-packages\scimap\tools\
cluster.py:174: FutureWarning:
```

```
In the future, the default backend for leiden will be igraph instead
of leidenalg.

 To achieve the future defaults please pass: flavor="igraph" and
n_iterations=2.  directed must also be False to work with igraph's
implementation.


AnnData object with n_obs × n_vars = 34162 × 30
    obs: 'merged_annotation', 'kmeans_1', 'kmeans_2', 'kmeans_3',
'kmeans_4', 'kmeans_5', 'kmeans_6', 'kmeans_7', 'kmeans_8',
'kmeans_9', 'kmeans_10', 'kmeans_11', 'kmeans_12', 'kmeans_13',
'kmeans_14', 'leiden_0.1'
```

Distance metric is Euclidean (it's the default).

ii.

```
# Count clusters obtained
leiden_labels = adata.obs['leiden_0.1'].astype(int)
n_leiden_clusters = leiden_labels.nunique()
print("Number of obtained leiden cluster:", n_leiden_clusters)
ll = pd.DataFrame(leiden_labels).value_counts().sort_index()
ll
```

```
Number of obtained leiden cluster: 9

leiden_0.1
0             7233
1             6133
2             4936
3             4482
4             3562
5             2843
6             2638
7             1667
8              668
Name: count, dtype: int64
```

iii.

```
# Extract labels
true_labels =
adata.obs['merged_annotation'].astype('category').cat.codes.values
ari_leiden = adjusted_rand_index(true_labels, leiden_labels)
print(f"ARI (Leiden res=0.1 vs merged_annotation): {ari_leiden:.4f}")
```

```
ARI (Leiden res=0.1 vs merged_annotation): -0.0000
```

```
C:\Users\Trish\AppData\Local\Temp\ipykernel_27356\3703409219.py:10:
RuntimeWarning:

overflow encountered in scalar multiply
```

The ARI value of 0 can mean that the clusters are random and independent.

C.

i.

```
# Compute UMAP first (scimap)
scimap.tl.umap(adata)
# Run K-means k=4
scimap.tl.cluster(adata, method='kmeans', k=4, label='kmeans_4_umap',
random_state=42, use_raw=False)
```

```
C:\Users\Trish\anaconda3\Lib\site-packages\umap\umap_.py:1952:
UserWarning:

n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.


Kmeans clustering

AnnData object with n_obs × n_vars = 34162 × 30
    obs: 'merged_annotation', 'kmeans_1', 'kmeans_2', 'kmeans_3',
'kmeans_4', 'kmeans_5', 'kmeans_6', 'kmeans_7', 'kmeans_8',
'kmeans_9', 'kmeans_10', 'kmeans_11', 'kmeans_12', 'kmeans_13',
'kmeans_14', 'leiden_0.1', 'kmeans_4_umap'
    obsm: 'umap'
```

ii.

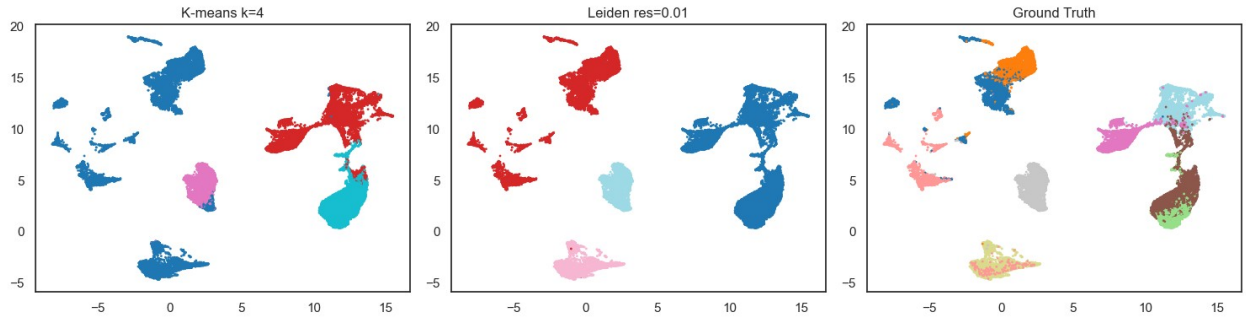
```
scimap.tl.cluster(adata, method='leiden', nearest_neighbors=20,
resolution=0.01,label='leiden_0.01', random_state=42, use_raw=False)
```

```
Leiden clustering

C:\Users\Trish\anaconda3\Lib\site-packages\scanpy\preprocessing\_pca\
__init__.py:384: ImplicitModificationWarning:

Setting element `.obsm['X_pca']` of view, initializing view as actual.


AnnData object with n_obs × n_vars = 34162 × 30
    obs: 'merged_annotation', 'kmeans_1', 'kmeans_2', 'kmeans_3',
'kmeans_4', 'kmeans_5', 'kmeans_6', 'kmeans_7', 'kmeans_8',
'kmeans_9', 'kmeans_10', 'kmeans_11', 'kmeans_12', 'kmeans_13',
```

```
'kmeans_14', 'leiden_0.1', 'kmeans_4_umap', 'leiden_0.01'
    obsm: 'umap'
```

UMAP and Leiden plots vs ground truth UMAP

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Kmeans = 4 Umap
k4 = adata.obs['kmeans_4'].astype(int).values
axes[0].scatter(adata.obsm['umap'][:,0], adata.obsm['umap'][:,1],
c=k4, cmap='tab10', s=1)
axes[0].set_title('K-means k=4')

# Leiden res=0.01 UMAP
leiden01 = adata.obs['leiden_0.01'].astype(int).values
axes[1].scatter(adata.obsm['umap'][:,0], adata.obsm['umap'][:,1],
c=leiden01, cmap='tab20', s=1)
axes[1].set_title('Leiden res=0.01')

# Ground truth UMAP
truth =
adata.obs['merged_annotation'].astype('category').cat.codes.values
axes[2].scatter(adata.obsm['umap'][:,0], adata.obsm['umap'][:,1],
c=truth, cmap='tab20', s=1)
axes[2].set_title('Ground Truth')

plt.tight_layout()
plt.show()
```



The potential disadvantage of K-means–based clustering methods in single-cell RNA-seq analysis is that cells can be forced into very specific clusters and underlying factors like DE GENES or specialized functions are ignored. When we look at the KMeans and Leiden plots side by side, we can see that in the Kmeans, there are some merging of clusters, but many cells are misclassified due to their approximate distance to the centroids, as compared to the Leiden plot. Thus, more cells are more likely misclassified as another cell type; their innate function are basically ignored due to proximity.

3. a. Find the Probability P(M=ACCTTA):

$0.8 \times 0.4 \times 0.05 \times 0.05 \times 0.8 \times 0.3$

$= 0.0001152$

b. Provide the Concensus Sequence corresponding to the PWM (find the sequence where P(M=m) is maximized)

	M_1	M_2	M_3	M_4	M_5	M_6
A	0.8	0.1	0	0.9	0	0.3
C	0	0.4	0.05	0.03	0.1	0.2
G	0.2	0	0.95	0.02	0.1	0.1
T	0	0.5	0	0.05	0.8	0.4

ATGATT

c. Given the logos, which best represents PWM & why.

Amongst the 3 logos, the best representative is logo 3. We can compare each logo to the PWM column by column. By doing so logo 2 is eliminated as it implies A+G are 0.5 when A posesses a stronger motiff, thus leaving 1 + 3 (based on column 1). Logo 1 is eliminated as in the third column, it implies G is 0.95 and T is 0.05, when the PWM says T is 0. Thus the only one correct is 3.

d. Which position of PWM has the greatest entropy?

$M_1 = -(0.8\log_2(0.8) + 0.2\log_2(0.2)) = 0.721928$

$M_2 = -(0.1\log_2(0.1) + 0.4\log_2(0.4) + 0.5\log_2(0.5)) = 1.36096$

$M_3 = -(0.05\log_2(0.05) + 0.95\log_2(0.95)) = 0.286397$

$M_5 = -(2(0.1\log_2(0.1)) + 0.8\log_2(0.8)) = 0.921928$

$M_4 = -(0.9\log_2(0.9) + 0.03\log_2(0.03) + 0.02\log_2(0.02) + 0.05\log_2(0.05)) = 0.617543$

$M_6 = -(0.3\log_2(0.3) + (0.2\log_2(0.2) + 0.1\log_2(0.1) + 0.4\log_2(0.4)) = 1.84644$

Position 6 has the highest entropy. Since entropy refers to disorder the larger the entropy, there is a decrease in conservation of information.

3. e. "X is a true instance of M if LLR > 0, or if S(X) > 0." Is this statistically sound, why or why not.

No, this is not statistically sound due to some issues. First, using 0 as a threshold means that there would be many false positives with this threshold being extremely low. Secondly, having LLR > 0 implicitly assumes that equal weight, thus ignoring the significance of motifs

f. describe a procedure to find the p-value.

p-value = $P(S \geq S(X) \mid H_0)$

H_0: X is drawn from the background distribution

S(X): observed LLR

First we set a significance level that determines the probability of getting a type I error. We then calculate our observed test statistic. (observed LLR score). We then generate the distribution of the background via simulation, then we calculate the probability of S per simulation. Finally we get the P-value of S by getting calculating the probability of getting said LLR or something more extreme, from our null distribution. We then set our found P-value against our level of significance to determine wether or not the motiff is present in that position.